

2.1 Graphs

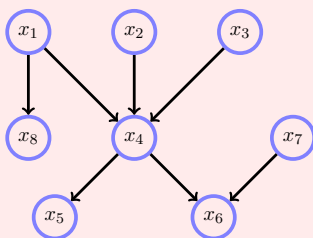
Definition 8 (Graph). A *graph* G consists of vertices (nodes) and edges (links) between the vertices. Edges may be directed (they have an arrow in a single direction) or undirected. A graph with all edges directed is called a *directed graph*, and one with all edges undirected is called an *undirected graph*.

Definition 9 (Path, Ancestors, Descendants). A *path* $A \mapsto B$ from node A to node B is a sequence of vertices $A_0 = A, A_1, \dots, A_{n-1}, A_n = B$, with (A_n, A_{n+1}) an edge in the graph, thereby connecting A to B . For a directed graph this means that a path is a sequence of nodes which when we follow the direction of the arrows leads us from A to B .

The vertices A such that $A \mapsto B$ and $B \not\mapsto A$ are the *ancestors* of B . The vertices B such that $A \mapsto B$ and $B \not\mapsto A$ are the *descendants* of A [168].

Definition 10 (Directed Acyclic Graph (DAG)). A DAG is a graph G with directed edges (arrows on each link) between the vertices (nodes) such that by following a path of vertices from one node to another along the direction of each edge no path will revisit a vertex. In a DAG the ancestors of B are those nodes who have a directed path ending at B . Conversely, the descendants of A are those nodes who have a directed path starting at A .

Definition 11 (Relationships in a DAG).



The *parents* of x_4 are $\text{pa}(x_4) = \{x_1, x_2, x_3\}$. The *children* of x_4 are $\text{ch}(x_4) = \{x_5, x_6\}$. The *family* of a node is itself and its parents. The *Markov blanket* of a node is itself, its parents, children and the parents of its children. In this case, the Markov blanket of x_4 is $x_1, x_2, \dots, x_7$.

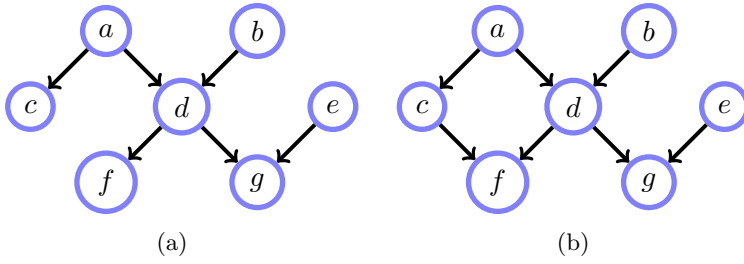
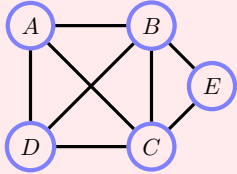


Figure 2.1: (a) Singly-Connected graph.
(b) Multiply-Connected graph.

Definition 12 (Undirected graph).

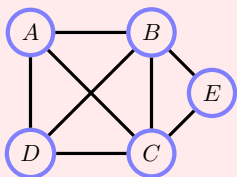


An undirected graph G consists of undirected edges between nodes.

Definition 13 (Neighbour). For an undirected graph G the neighbours of x , $ne(x)$ are those nodes directly connected to x .

Definition 14 (Connected graph). An undirected graph is *connected* if there is a path between every set of vertices (*i.e.* there are no isolated islands). For a graph which is not connected, the connected components are those subgraphs which are connected.

Definition 15 (Clique).



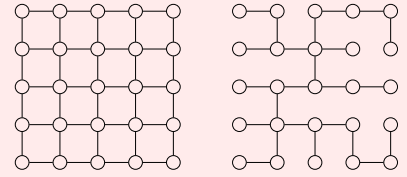
Given an undirected graph, a clique is a maximally connected subset of vertices. All the members of the clique are connected to each other; furthermore there is no larger clique that can be made from a clique. For example this graph has two cliques, $\mathcal{C}_1 = \{A, B, C, D\}$ and $\mathcal{C}_2 = \{B, C, E\}$. Whilst A, B, C are fully connected, this is a non-maximal clique since there is a larger fully connected set, A, B, C, D that contains this. A non-maximal clique is sometimes called a *cliquo*.

Definition 16 (Singly-Connected Graph). A graph is *singly-connected* if there is only one path from a vertex a to another vertex b . Otherwise the graph is *multiply-connected*. This definition applies regardless of whether or not the edges in the graph are directed. An alternative name for a singly-connected graph is a *tree*. A multiply-connected graph is also called *loopy*.

2.1.1 Spanning tree

Definition 17 (Spanning Tree).

A spanning tree of an undirected graph G is a singly-connected subset of the existing edges such that the resulting singly-connected graph covers all vertices of G . On the right is a graph and an associated spanning tree. A maximum weight spanning tree is a spanning tree such that the sum of all weights on the edges of the tree is larger than for any other spanning tree of G .



Finding the maximal weight spanning tree

A simple algorithm to find a spanning tree with maximal weight is as follows: Start by picking the edge with the largest weight and add this to the edge set. Then pick the next candidate edge which has the largest weight and add this to the edge set – if this results in an edge set with cycles, then reject the candidate edge and find the next largest edge weight. Note that there may be more than one maximal weight spanning tree.

2.2 Numerically Encoding Graphs

To express the structure of GMs we need to numerically encode the links on the graphs. For a graph of N vertices, we can describe the graph structure in various equivalent ways.

2.2.1 Edge list

As the name suggests, an *edge list* simply lists which vertex-vertex pairs are in the graph. For fig(2.2a), an edge list is $L = \{(1, 2), (2, 1), (1, 3), (3, 1), (2, 3), (3, 2), (2, 4), (4, 2), (3, 4), (4, 3)\}$ where an undirected edge is represented by a bidirectional edge.

2.2.2 Adjacency matrix

An alternative is to use an *adjacency matrix*

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} \quad (2.2.1)$$

where $A_{ij} = 1$ if there is an edge from variable i to j in the graph, and 0 otherwise. Some authors include self-connections in this definition. An undirected graph has a symmetric adjacency matrix.

Provided that the vertices are labelled in *ancestral order* (parents always come before children) a directed graph fig(2.2b) can be represented as a triangular adjacency matrix:

$$\mathbf{T} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (2.2.2)$$

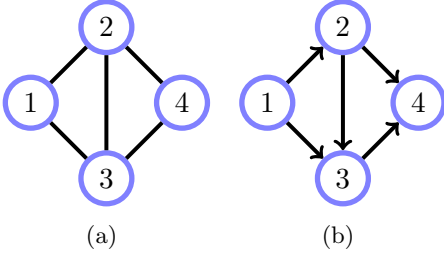


Figure 2.2: (a): An undirected graph can be represented as a symmetric adjacency matrix. (b): A directed graph with vertices labelled in ancestral order corresponds to a triangular adjacency matrix.

Adjacency matrix powers

For an $N \times N$ adjacency matrix $\mathbf{A}$, powers of the adjacency matrix $[\mathbf{A}^k]_{ij}$ specify how many paths there are from node i to node j in k edge hops.

If we include 1's on the diagonal of $\mathbf{A}$ then $[\mathbf{A}^N]_{ij}$ is non-zero when there is a path connecting j to i in the graph. If $\mathbf{A}$ corresponds to a DAG the non-zero entries of the j^{th} row of $[\mathbf{A}^N]$ correspond to the descendants of node j .

2.2.3 Clique matrix

For an undirected graph with N vertices and maximal cliques $C_1, \dots, C_K$ a clique matrix is an $n \times K$ matrix in which each column c_k has zeros except for ones on entries describing the clique. A cliquo matrix relaxes the constraint that cliques are required to be maximal¹. For example

$$\mathbf{C} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} \quad (2.2.3)$$

is a clique matrix for fig(2.2a). A cliquo matrix containing only two-dimensional maximal cliques is called an *incidence matrix*. For example

$$\mathbf{C}_{inc} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix} \quad (2.2.4)$$

is an incidence matrix for fig(2.2b).

It is straightforward to show that $\mathbf{C}_{inc} \mathbf{C}_{inc}^T$ is equal to the adjacency matrix except that the diagonals now contain the *degree* of each vertex (the number of edges it touches). Similarly, for any cliquo matrix the diagonal entry of $[\mathbf{C}\mathbf{C}^T]_{ii}$ expresses the number of cliquos (columns) that vertex i occurs in. Off diagonal elements $[\mathbf{C}\mathbf{C}^T]_{ij}$ contain the number of cliquos that vertices i and j jointly inhabit.

Remark 2 (Graph Confusions). Graphs are widely used, but differ markedly in what they represent. Two potential pitfalls are described below.

State Transition Diagrams Such graphical representations are common in Markov Chains and Finite State Automata. A set of states is written as set of nodes(vertices) of a graph, and a directed edge between node i and node j (with an associated weight p_{ij}) represents that a transition from state i to state j can occur with probability p_{ij} . From the graphical models perspective we would simply write down a directed graph $x(t) \rightarrow x(t+1)$ to represent this Markov Chain. The state-transition diagram simply provides a graphical description of the conditional probability table $p(x(t+1)|x(t))$.

¹The term ‘cliquo’ for a non-maximal clique is attributed to Julian Besag.

Neural Networks Neural networks also have vertices and edges. In general, however, neural networks are graphical representations of *functions*, whereas as graphical models are representations of distributions (a richer formalism). Neural networks (or any other parametric description) may be used to represent the conditional probability tables, as in sigmoid belief networks[204].

2.3 Code

2.3.1 Utility routines

`ancestors.m`: Find the ancestors of a node in a DAG

`edges.m`: Edge list from an adjacency matrix

`ancestralorder.m`: Ancestral order from a DAG

`connectedComponents.m`: Connected Components

`parents.m`: Parents of a node given an adjacency matrix

`children.m`: Children of a node given an adjacency matrix

`neigh.m`: Neighbours of a node given an adjacency matrix

A connected graph is a tree if the number of edges plus 1 is equal to the number of nodes. However, for a possibly disconnected graph this is not the case. The code below deals with the possibly disconnected case. The routine is based on the observation that any singly-connected graph must always possess a simplicial node (a leaf node) which can be eliminated to reveal a smaller singly-connected graph.

`istree.m`: If graph is singly connected return 1 and elimination sequence

`spantree.m`: Return a spanning tree from an ordered edge list

`singleparenttree.m`: Find a directed tree with at most one parent from an undirected tree

Additional routines for basic manipulations in graphs are given at the end of chapter(6).

2.4 Exercises

Exercise 17. Consider an adjacency matrix $\mathbf{A}$ with elements $[\mathbf{A}]_{ij} = 1$ if one can reach state i from state j in one timestep, and 0 otherwise. Show that the matrix $[\mathbf{A}^k]_{ij}$ represents the number of paths that lead from state j to i in k timesteps. Hence derive an algorithm that will find the minimum number of steps to get from state j to state i .

Exercise 18. For an $N \times N$ symmetric adjacency matrix $\mathbf{A}$, describe an algorithm to find the connected components. You may wish to examine `connectedComponents.m`.

Exercise 19. Show that for a connected graph that is singly-connected, the number of edges E must be equal to the number of vertices minus 1, $E = V - 1$. Give an example graph with $E = V - 1$ that is not singly-connected.

